

Thank you for your interest! Here's a detailed overview of our project — **SafeReach: An Inclusive AI-Enabled SOS & Assistive Safety Platform.**

Key Features:

- **One-Tap SOS Alert** — A large, accessible SOS button for instant emergency triggering
- **Shake Detection** — Rapidly shaking the phone triggers an emergency alert automatically
- **Fall Detection** — AI-based accelerometer analysis detects sudden falls and auto-triggers alerts
- **Voice Activation** — Custom wake-word detection (e.g., "Help me") for hands-free SOS
- **Accessibility-First Design** — Supports Pictogram mode, High Contrast mode, and Text-to-Speech for users with visual, hearing, or cognitive disabilities
- **Auto SMS Dispatch** — Sends emergency SMS with live GPS location (Google Maps link) and user's accessibility needs to all saved contacts in the background
- **QR Code Safety** — QR-based identification for quick responder access to the user's medical/accessibility profile
- **Incident History & Logging** — Complete timeline of past emergencies stored locally for review
- **Real-Time Location Tracking** — Live GPS tracking shared during active emergencies
- **Admin/Responder Dashboard** — A separate web dashboard (React + Vite) with live alerts, analytics, volunteer management, and QR code management for institutional responders like campus security

User Flow:

1. **Onboarding** → User sets up their profile, accessibility preferences, and emergency contacts
2. **Home Screen** → Central SOS button always visible, with live location badge
3. **Trigger** → Emergency activated via button tap, shake, fall detection, or voice command
4. **Countdown** → A 5-second cancellable countdown with haptic feedback and TTS warning (prevents false triggers)
5. **Active Alert** → GPS location acquired → SMS sent to all emergency contacts with map link + accessibility info → Live timer starts
6. **Escalation** → If no contact acknowledges, the system auto-escalates
7. **Resolution** → User or responder marks "Safe" → Incident logged → App returns to idle

Tech Stack:

- **Mobile App:** Flutter & Dart with Riverpod (state management), Hive (local DB), GoRouter (navigation)
- **Backend:** Firebase (Auth, Firestore, Cloud Messaging)
- **Dashboard:** React.js, Vite, Recharts, Leaflet (maps), React Router

- **Sensors & APIs:** Accelerometer (sensors_plus), Geolocator, Speech-to-Text, Flutter TTS, Android Telephony API

Challenges We Faced:

- **Background SMS on iOS** — Apple's strict sandboxing doesn't allow background SMS like Android, so we had to implement a fallback using the system share sheet
- **Battery Optimization** — Continuous accelerometer polling and voice wake-word detection drain battery fast; we had to balance detection accuracy with adaptive sampling
- **Offline-First Architecture** — Designing the app to work fully offline (using local Hive storage + SMS) while still syncing with Firebase when connected was complex
- **Fall Detection Accuracy** — Differentiating a real fall from normal phone drops required careful G-force threshold tuning and multi-axis analysis
- **Accessibility Compliance** — Ensuring the app is genuinely usable for users with different disabilities (Deaf, Blind, Cognitive) went beyond standard accessibility — we built custom adaptive UI modes